COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Unit Convenor)
ptsai@swin.edu.au, EN508d

SWINBURNE UNIVERSITY OF TECHNOLOGY

Week 7

Common MapReduce Algorithms

COMMON MAPREDUCE ALGORITHMS

Summary

MapReduce jobs tend to be relatively short in terms of lines of code.

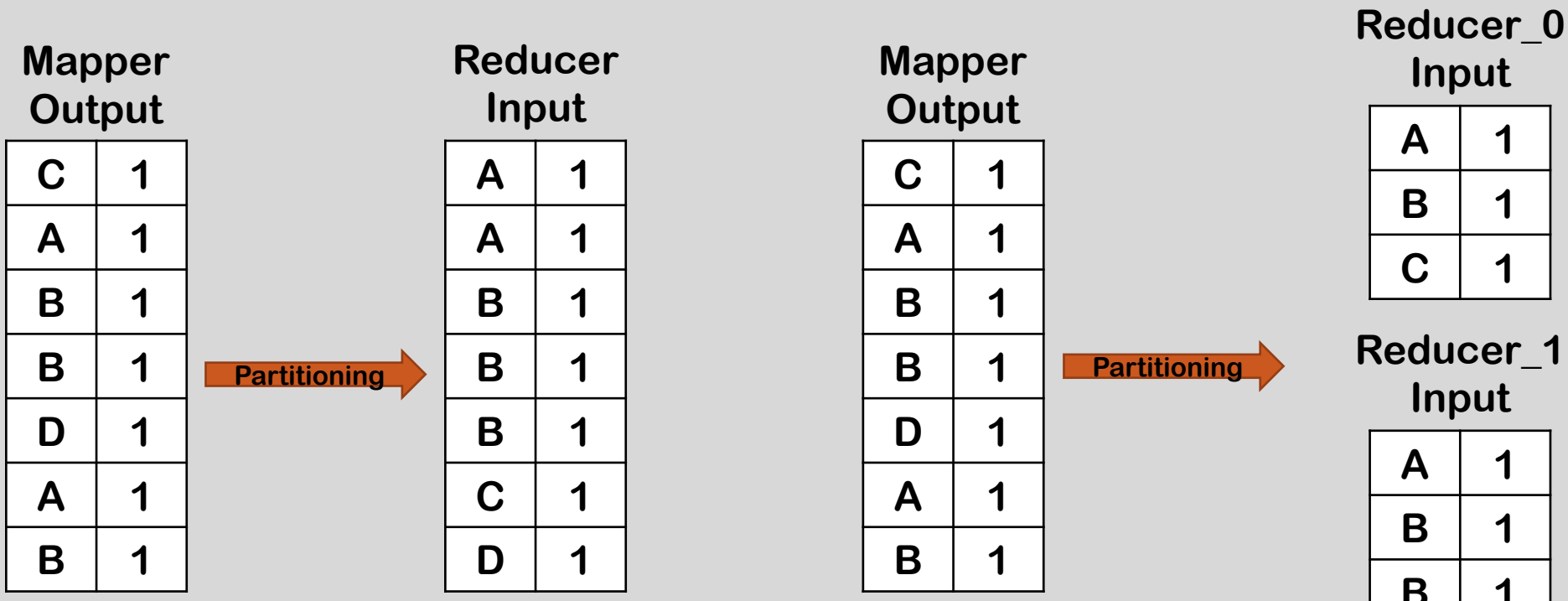
It is typical to combine multiple small MapReduce jobs together in a single workflow. (Can be achieved by using Oozie)

Many of your MapReduce jobs use very similar codes.

We will present some very common MapReduce algorithms in this lecture. These algorithms are frequently the basis for more complex MapReduce jobs.

Sorting

- MapReduce is very well suited to sorting large data sets.
- Recall: keys are passed to the Reducer in sorted order.



Sorting

- For multiple reducers, you need to choose a partitioning function such that if $k_1 < k_2$, $partition(k_1) \leq partition(k_2)$.

Mapper Output

C	1
A	1
B	1
B	1
D	1
A	1
B	1

Partitioning

Reducer Input

A	1
A	1
B	1
B	1
B	1
C	1
D	1

Mapper Output

C	1
A	1
B	1
B	1
D	1
A	1
B	1

Partitioning

Reducer_0 Input

A	1
A	1
B	1
B	1
B	1

Reducer_1 Input

C	1
D	1

Sorting as a Speed Test of Hadoop

Sorting is frequently used as a speed test for a Hadoop cluster.

- Mapper and Reducer are trivial.
  - Therefore sorting is effectively testing the Hadoop framework's I/O.

Good way to measure the increase in performance if you enlarge your cluster.

- Run and time a sort job before and after you add more nodes.
- *TeraSort* is one of the sample jobs provided with Hadoop.
  - It measures the time to sort 1 TB of randomly generated data.

https://www.youtube.com/watch?v=XbUPlbYxT8g&feature=emb_imp_woyt

Searching

Input

Assume the input is a set of files containing lines of text.

Parameter passing

Assume the Mapper has been passed the pattern for which to search as a special parameter.

Algorithm:

Mapper compares the line against the pattern.

If the pattern matches, output *(line, content)* or *(filename + line, content)*, etc.

If the pattern does not match, Mapper outputs nothing.

Reducer is the Identity Reducer

- Just outputs each intermediate key.

Indexing

Input

- Assume the input is **a set of files** containing lines of text.

Key is the byte offset of the line, value is the line itself.

We can retrieve the name of the file using the Context object.

Inverted Index

It creates a map for terms to a list of identifiers.

Huge dataset usually builds and stores an Inverted Index so that the retrieval process is efficient.

Building inverted index is a straight-forward application of MapReduce framework.

Inverted Index Algorithm

Mapper

- For each word in the line, emit (word, filename)

Reducer

- Identity function
  - Collect together all values for a given key (all filenames for a particular word)
  - Emit (word, filename_list)

Inverted Index: Example

I am a student at SUT.

Doc 1

I currently study for Big Data Architecture and Applications.

Doc 2

We have developed several MapReduce programs in the unit.

Doc 3

word list

.
a
am
and
applications
architecture
at
big
currently
data
developed
for
have
i
in
mapreduce
programs
several
student
study
sut
the
unit
we

ID	Term	Doc
1	.	1,2,3
2	a	1
3	am	1
4	and	2
5	applications	2
6	architecture	2
7	at	1
8	big	2
9	currently	2
10	data	2
11	developed	3
12	for	2
13	have	3
14	i	1,2
15	in	3
16	mapreduce	3
17	programs	3
18	several	3
19	student	1
20	study	2
21	sut	1
22	the	3
23	unit	3
24	we	3

Example of Inverted Index

Task

- Given a set of employee information documents. Find the document in which employee information is available based on the first name of the employee.

Target

- Building an inverted index in the form of *FirstName → FileName (or FileNameList)*.

Assume the sample data files are:

- E_info_1.csv
- E_info_2.csv
- E_info_3.csv
- Data format: First Name, Last Name, Title, …

	A	B	C	D	E	F	G	H	I
1	ALLISON	PAUL W	LIEUTENA	FIRE	F	Salary		107790	
2	BRUNO	KEVIN D	SERGEANT	POLICE	F	Salary		104628	
3	COOPER	JOHN E	LIEUTENA	FIRE	F	Salary		114324	
4	CRESPO	VILMA I	STAFF ASS	LAW	F	Salary		76932	
5	DOLAN	ROBERT J	SERGEANT	POLICE	F	Salary		111474	
6	DUBERT	TOMASZ	PARAMED	FIRE	F	Salary		91080	
7	EDWARDS	TIM P	LIEUTENA	FIRE	F	Salary		114846	
8	ELKINS	ERIC J	SERGEANT	POLICE	F	Salary		104628	
9	ESTRADA	LUIS F	POLICE OF	POLICE	F	Salary		96060	
10	EWING	MARIE A	CLERK III	POLICE	F	Salary		53076	
11	FINN	SEAN P	FIREFIGHT	FIRE	F	Salary		87006	
12	FITCH	JORDAN M	LAW CLER	LAW	F	Hourly	35		14.51
13	FREELON	CHERYL N	POLICE OF	POLICE	F	Salary		96060	
14	GRAFF	KEVIN M	SERGEANT	POLICE	F	Salary		111474	
15	GUNN	ISSAC J	FIREFIGHT	FIRE	F	Salary		80868	
16	HARRIS	DANIEL J	POLICE OF	POLICE	F	Salary		48078	
17	HATTULA	CARL J	SERGEANT	POLICE	F	Salary		107988	
18	HILL	LATINA P	PARAMED	FIRE	F	Salary		72510	
19	HOLLER	JOEL P	SERGEANT	POLICE	F	Salary		104628	
20	HROMA JF	JOHN	SERGEANT	POLICE	F	Salary		107988	
21	JACKSON	ERIC	SERGEANT	POLICE	F	Salary		107988	
22	JOYCE	SEAN G	CAPTAIN	POLICE	F	Salary		132876	

https://timepasstechies.com/map-reduce-inverted-index-sample/

MAPPER CODE

```
1.  import java.io.IOException;
2.  import org.apache.hadoop.io.Text;
3.  import org.apache.hadoop.mapreduce.Mapper;
4.  import org.apache.hadoop.mapreduce.lib.input.FileSplit;
5.
6.  public class InvertedIndexNameMapper extends Mapper<Object, Text, Text, Text> {
7.
8.  private Text nameKey = new Text();
9.  private Text fileNameValue = new Text();
10.
11. @Override
12. public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
13. String data = value.toString();
14. String[] field = data.split(",", -1);
15. String firstName = null;
16.
17. if (null != field && field.length == 9 && field[0].length() > 0) {
18. firstName=field[0];
19. String fileName = ((FileSplit) context.getInputSplit()).getPath().getName();
20. nameKey.set(firstName);
21. fileNameValue.set(fileName);
22. context.write(nameKey, fileNameValue);
23. }
24.
25. }
26.
27. }
```

REDUCER CODE

```java
import java.io.IOException;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class InvertedIndexNameReducer extends Reducer<Text, Text, Text, Text> {

private Text result = new Text();

public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {

StringBuilder sb = new StringBuilder();
boolean first = true;
for (Text value : values) {

if (first) {
first = false;
} else {
sb.append(" ");
}

if (sb.lastIndexOf(value.toString()) < 0) {
sb.append(value.toString());
}

}
result.set(sb.toString());

context.write(key, result);

}

}
```

DRIVER CODE

```java
import java.io.File;
import org.apache.commons.io.FileUtils;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class DriverInvertedIndex {

public static void main(String[] args) throws Exception {

/*
* I have used my local path in windows change the path as per your
* local machine
*/

args = new String[] { "Replace this string with Input Path location",
"Replace this string with output Path location" };

/* delete the output directory before running the job */

FileUtils.deleteDirectory(new File(args[1]));

/* set the hadoop system parameter */

System.setProperty("hadoop.home.dir", "Replace this string with hadoop home directory location");

if (args.length != 2) {
System.err.println("Please specify the input and output path");
System.exit(-1);
}

Configuration conf = ConfigurationFactory.getInstance();
Job job = Job.getInstance(conf);
job.setJarByClass(DriverInvertedIndex.class);
job.setJobName("Find_Average_Salary");
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
job.setMapperClass(InvertedIndexNameMapper.class);
job.setReducerClass(InvertedIndexNameReducer.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);
System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

Example Result

```
AARON    employee_info_1.csv   employee_info_2.csv

ABAD JR  employee_info_1.csv

ABARCA   employee_info_1.csv
```

COMPUTING TERM FREQUENCY – INVERSE DOCUMENT FREQUENCY (TF-IDF)

Term Frequency – Inverse Document Frequency

Answers the question “How important this term is in a document?”

Known as a term weighting function

- Assigns a score (weight) to each term (word) in a document.

Very commonly used in text processing and search.

Has many applications in data mining.

Why Use TF-IDF?

- Merely counting the number of occurrences of a word in a document is not a good enough measure of its relevance.
  - If the word appears in many other documents, it is probably less relevant.
  - Some words appear too frequently in all documents to be relevant:
    - Known as “stopwords”
    - Ex: a, the, this, to, from, etc.
- TF-IDF considers both the frequency of a word in a given document and the number of documents which contain the word.

TF-IDF Data Mining Example

Scenario

- In a music recommendation system, given many user's music libraries, provide "you may also like" suggestions.

If user A and user B have similar libraries, user A may like an artist in user B's library.

- But some artists will appear in almost everyone's library, and should therefore be ignored when making recommendations.
- Similar as in Natural Language Processing (NLP), the most common words are meaningless such as "a, the, this, etc."

TF-IDF Formally Defined

Term Frequency (TF)

- Number of times a term appears in a document. (i.e., the count)

Inverse Document Frequency (IDF)

- $IDF = \log_a\left(\frac{N}{n}\right)$, where N is the total number of documents, n stands for the number of documents that contain a term, a can be 2 or 10 or any number.

TF-IDF

- TF $\times$ IDF

TF-IDF Example

- **Assume you have a term for comparing the similarity**
  - $D_0 = \{terms\}$
- **You'll be able to create a TF table for all terms against all documents ($TF_{i,j}$ and DF_i). where $TF_{i,j}$ is the count of $term_i$ appearing in document j and DF_i is the number of documents containing the i^{th} term.**

	D_1	D_2	$\cdots$	D_N	DF
$term_1$	$TF_{1,1}$	$TF_{1,2}$	$\cdots$	$TF_{1,N}$	DF_1
$term_2$	$TF_{2,1}$	$TF_{2,2}$	$\cdots$	$TF_{2,N}$	DF_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
$term_M$	$TF_{M,1}$	$TF_{M,2}$	$\cdots$	$TF_{M,N}$	DF_M

  - $IDF = \log\left(\frac{N}{DF}\right)$
- **The weight for each term:**
  - $Weight(TF_{i,j}, DOC_j) = TF_{i,j} \times \log\left(\frac{N}{DF_i}\right)$

Computing TF-IDF

Required Information

- Number of times t appears in a document.
  (N values if you have N documents.)
- Number of documents that contains t.
  (One value for each term.)
- Total number of documents.
  (One value: N)

Computing TF-IDF using MapReduce

Overview of algorithm: 3 MapReduce Jobs

- Job 1: compute term frequencies.
- Job 2: compute number of documents each word occurs in
- Job 3: compute TF-IDF

Notation in following slides:

- *docid* = a unique ID for each document
- *contents* = the complete text of each document
- *N* = total number of documents
- *term* = a term (word) found in the document
- *tf* = term frequency
- *n* = number of documents a term appears in

Note that real-world systems typically perform “stemming” on terms.

- Removal of plurals, tense, possessives, etc.

Computing TF-IDF: Job 1 – Compute *tf*

Mapper

- Input: (doc_id, contents)
  - For each term in the document, generate a (term, doc_id) pair.
- Output: ((term, doc_id), 1)

Reducer

- Sums counts for word in document.
- Output: ((term, doc_id), *tf*)

We can add a Combiner, which will use the same code as the Reducer.

Computing TF-IDF: Job 2 – Compute *n*

Mapper

- Input: ((term, doc_id), *tf*)
- Output: (term,(doc_id, *tf*,1))

Reducer

- Sums 1s to comput *n* (number of documents containing term)
- Note that we need to buffer (doc_id, *tf*) pairs while we are doing this for later use.
- Output: ((term, doc_id), (*tf, n*))

Computing TF-IDF: Job 3 – Compute TF-IDF

Mapper

- Input: ((term, doc_id), (*tf*, *n*))
- N (the total number of documents) is known (easy to find)
- Output: ((term, doc_id),TFxIDF)

Reducer

- The identity function

Computing TF-IDF: Working at Scale

In Job 2, we need to buffer (doc_id, *tf*) pairs counts while summing 1's (for computing *n*).

- Possible issue: pairs may not fit in the memory.
- Ex: in how many documents does the word "the" occur?

Possible solutions

- Ignore very-high-frequency words.
- Write out intermediate data to a file.
- Use another MapReduce pass.

TF-IDF Related Considerations

Several small jobs add up to full algorithm

- Thinking in MapReduce often means decomposing a complex algorithm into a sequence of smaller jobs.

Beware of memory usage for large amounts of data.

- Anytime when you need to buffer data, there is a potential scalability bottleneck.

CALCULATING WORD CO-OCCURRENCE

Why Using Word Co-Occurrence?

Word co-occurrence measures the frequency with which two words appear close to each other in a corpus of documents.

- Different distance definitions result in different results of "close" measurement.

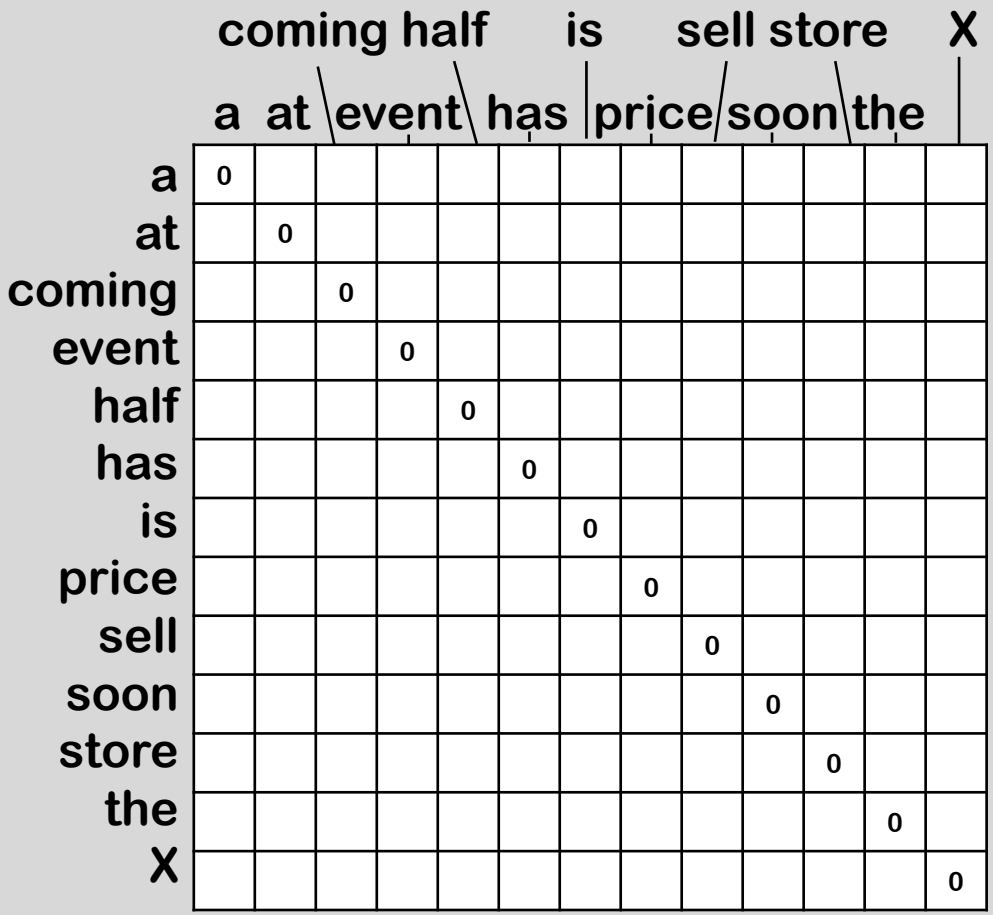
This is at the heart of many data-mining techniques.

- Provides results for "people who did this, also do that."
- Examples:
  - Shopping recommendations.
  - Credit risk analysis.
  - Identifying "people of interest."

Word Co-Occurrence Example

The half price sell is coming at store X.

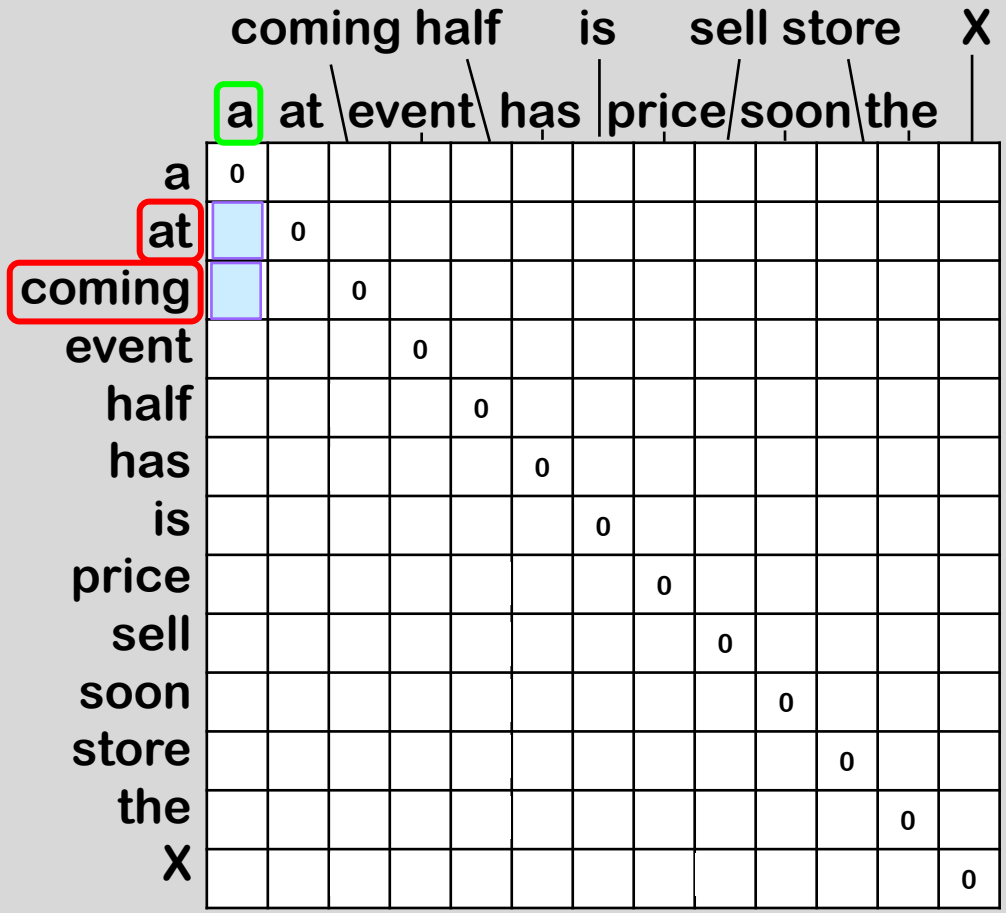
Store X has a half price sell event coming soon.



Word Co-Occurrence Example

The half price sell is coming at store X.

Store X has a half price sell event coming soon.



Word Co-Occurrence Example

The half price sell is coming at store X.

Store X has a half price sell event coming soon.

	coming	half		is		sell	store		X
	a	at	event	has	price	soon	the		
a	0								
at	0	0							
coming	1	0	0						
event	1	0	1	0					
half	1	1	2	1	0				
has	1	0	1	1	1	0			
is	0	1	1	0	1	0	0		
price	1	1	2	1	2	1	1	0	
sell	1	1	1	1	2	1	1	2	0
soon	1	0	1	1	1	1	0	1	1
store	1	1	2	1	2	1	1	2	2
the	0	1	1	0	1	0	1	1	1
X	1	1	2	1	2	1	1	2	2

Word Co-Occurrence Example

The half price sell is coming at store X.

Store X has a half price sell event coming soon.

	a	at	coming	event	half	has	is	price	sell	soon	store	the	X
a	0												
at	0	0											
coming	1	0	0										
event	1	0	1	0									
half	1	1	2	1	0								
has	1	0	1	1	1	0							
is	0	1	1	0	1	0	0						
price	1	1	2	1	2	1	1	0					
sell	1	1	1	1	2	1	1	2	0				
soon	1	0	1	1	1	1	0	1	1	0			
store	1	1	2	1	2	1	1	2	2	1	0		
the	0	1	1	0	1	0	1	1	1	0	1	0	
X	1	1	2	1	2	1	1	2	2	1	2	1	0

Word Co-Occurrence Example

The half price sell is coming at store X.

Store X has a half price sell event coming soon.

A half price sell is at store X.

	coming	half		is		sell	store		X
	a	at	event	has	price	soon	the		
a	0								
at	0	0							
coming	1	0	0						
event	1	0	1	0					
half	2	2	2	1	0				
has	1	0	1	1	1	0			
is	1	2	1	0	2	0	0		
price	2	2	2	1	3	1	2	0	
sell	2	2	1	1	3	1	2	3	0
soon	1	0	1	1	1	1	0	1	1
store	2	2	2	1	3	1	2	3	3
the	0	1	1	0	1	0	1	1	1
X	2	2	2	1	3	1	2	3	3

Word Co-Occurrence Example

The half price sell is coming at store X.

Store X has a half price sell event coming soon.

A half price sell is at store X.

	a	at	coming	event	half	has	is	price	sell	soon	store	the	X
a	0												
at	0	0											
coming	1	0	0										
event	1	0	1	0									
half	2	2	2	1	0								
has	1	0	1	1	1	0							
is	1	2	1	0	2	0	0						
price	2	2	2	1	3	1	2	0					
sell	2	2	1	1	3	1	2	3	0				
soon	1	0	1	1	1	1	0	1	1	0			
store	2	2	2	1	3	1	2	3	3	1	0		
the	0	1	1	0	1	0	1	1	1	0	1	0	
X	2	2	2	1	3	1	2	3	3	1	3	1	0

Algorithm for Word Co-Occurrence

- **Mapper**

```
Map (doc_id a, doc b) {
  foreach w in b do {
    foreach u near w do {
      emit(pair(w, u), 1)
    }
  }
}
```

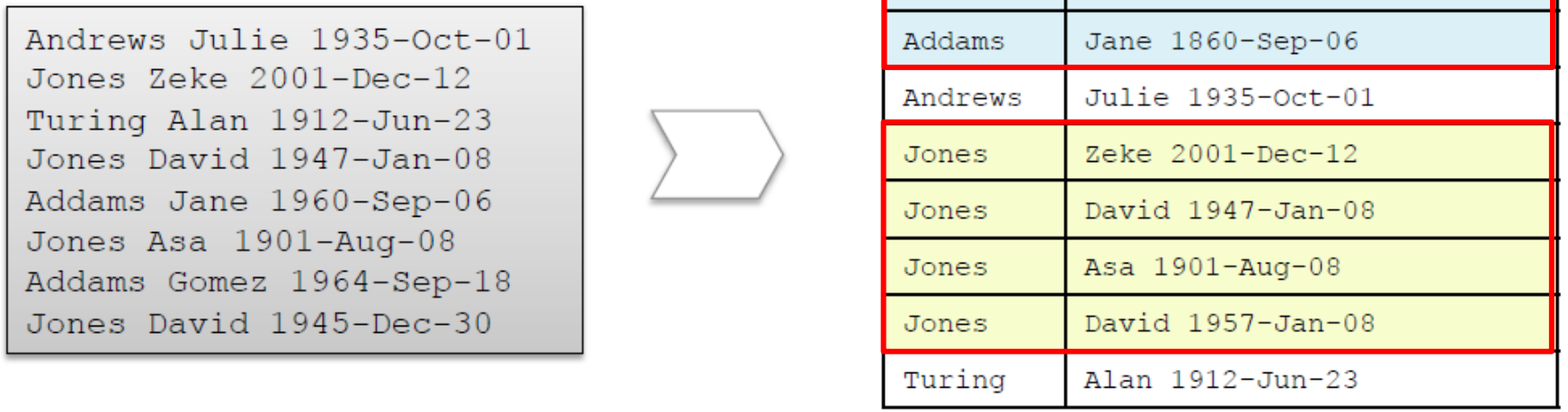
- **Reducer**

```
Reduce (pair p, Interator counts) {
  s = 0
  foreach c in counts do {
    s += c
  }
  emit(p, s)
}
```

PERFORMING A SECONDARY SORT

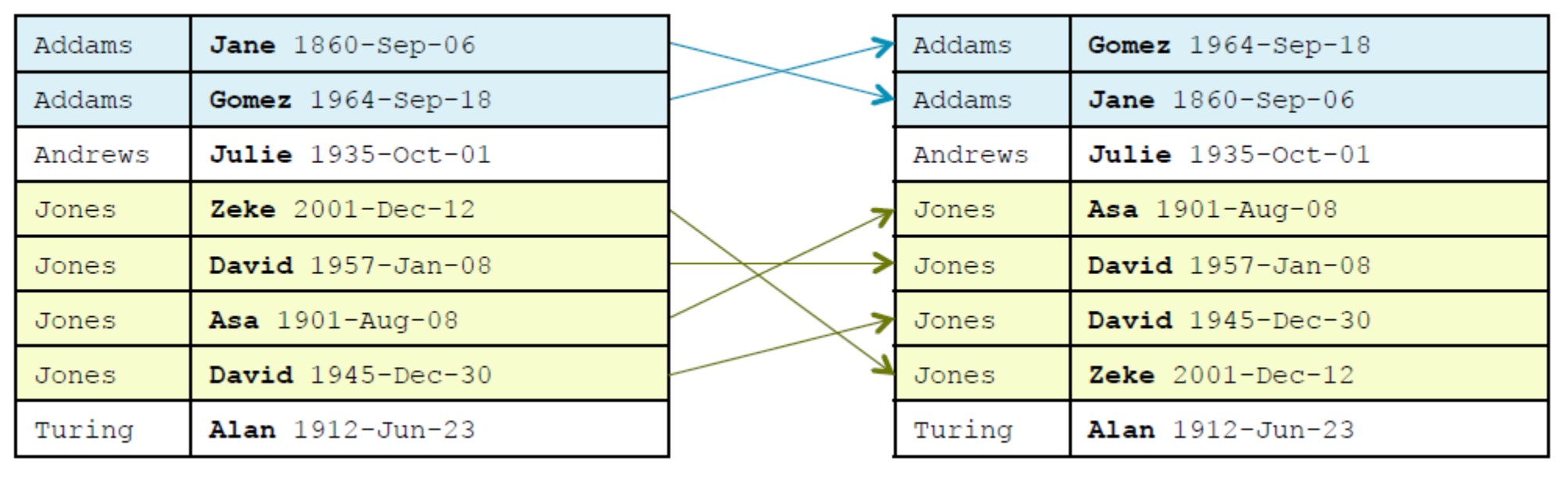
Why We Need a Secondary Sort?

- By default, the keys are passed to the Reducer in sorted order.
- The list of values for a particular key is NOT sorted.
  - Order may well change between different runs of the MapReduce job.



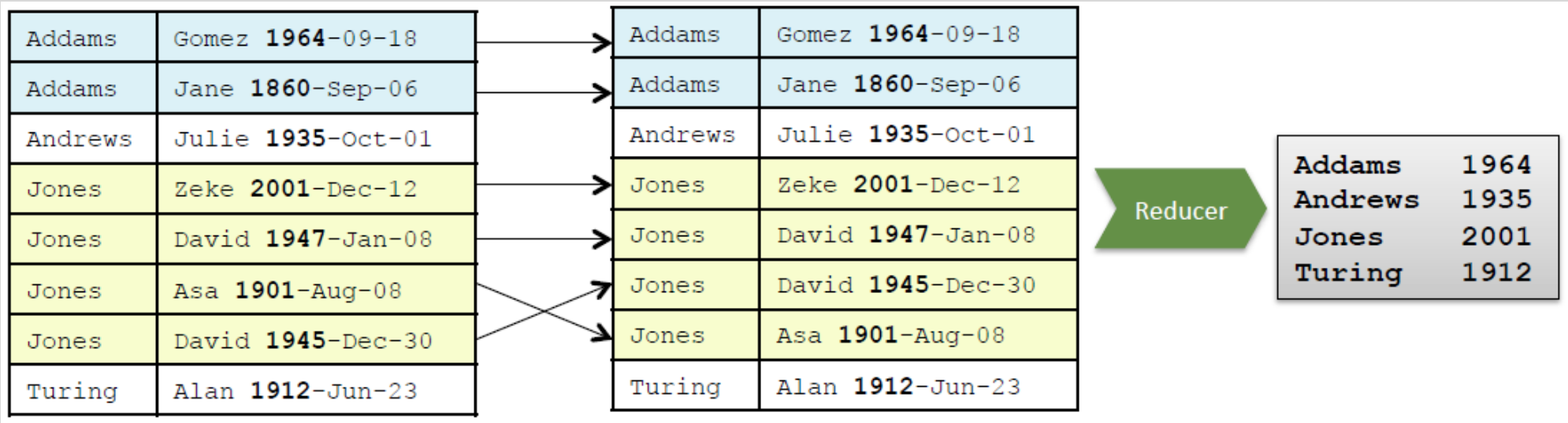
Why We Need a Secondary Sort?

- Sometimes a job needs to receive the values for a particular key in a sorted order. – This is known as a **secondary sort**.
- Example:
  - Sort by Last Name, then First Name



Why We Need a Secondary Sort?

- **Example: Find the latest birth year for each surname in a list.**
- **Naïve solution:**
  - **Reducer loops through all values, keeping track of the latest year.**
  - **Finally, emit the latest year. (Not a very good idea.)**
- **Better solution:**
  - **Pass the values sorted by year in descending order to the Reducer, which can then just emit the first value.**



Secondary Sort – Implementation [Composite Keys]

- To implement a secondary sort, the intermediate key should be a composite of the "actual" (natural) key and the value.
- Implement a mapper to construct composite keys.

```
let map(k, v) =
    emit(new Pair(v.getPrimaryKey(), v.getSecondaryKey)), v)
```

```
Jones Zeke 2001-Dec-12
Turing Alan 1912-Jun-23
Jones David 1947-Jan-08
Addams Jane 1860-Sep-06
Jones Asa 1901-Aug-08
Addams Gomez 1964-Sep-18
Jones David 1945-Dec-30
```

Mapper

Jones#2001	Jones Zeke 2001-Dec-12
Turing#1912	Turing Alan 1912-Jun-23
Jones#1947	Jones David 1947-Jan-08
Addams#1860	Addams Jane 1860-Sep-06
Jones#1901	Jones Asa 1901-Aug-08
Addams#1964	Addams Gomez 1964-Sep-18
Jones#1945	Jones David 1945-Dec-30

Secondary Sort – Implementation [Partitioning Composite Keys]

- Create a custom **partitioner** by using natural key to determine which reducer to send the key to.

```
let getPartition(Pair k, Text v, int numReducers) =
    return(k.getPrimaryKey().hashCode() % numReducers)
```

Jones#1947	Jones David 1947-Jan-08
Addams#1860	Addams Jane 1860-Sep-06
Jones#1901	Jones Asa 1901-Aug-08
Addams#1964	Addams Gomez 1964-Sep-18
Jones#1945	Jones David 1945-Dec-30

Partitioner

Partition 0	
Jones#1947	Jones David 1947-Jan-08
Jones#1901	Jones Asa 1901-Aug-08
Jones#1945	Jones David 1945-Dec-30

Partition 1	
Addams#1860	Addams Jane 1860-Sep-06
Addams#1964	Addams Gomez 1964-Sep-18

Secondary Sort – Implementation [Sorting Composite Keys]

- **Comparator** classes are classes that compare objects.

`compare(A,B)` returns:

  - 1 if A>B
  - 0 if A=B
  - -1 if A<B

- Custom comparators can be used to sort composite keys.
  - extend WritableComparator
  - Override int compare()
- Two comparators are required:
  - Sort Comparator
  - Group Comparator

Secondary Sort – Implementation [Sorting Comparator]

Sorting Comparator

- Sorts the input to the Reducer.
- Uses the full composite key: compares natural key first; if equal, compares secondary key.

```
let compare(Pair k1, Pair k2) =
    compare k1.getPrimaryKey(), k2.getPrimaryKey()
    if equal
        compare k1.getSecondaryKey(), k2.getSecondaryKey()
```

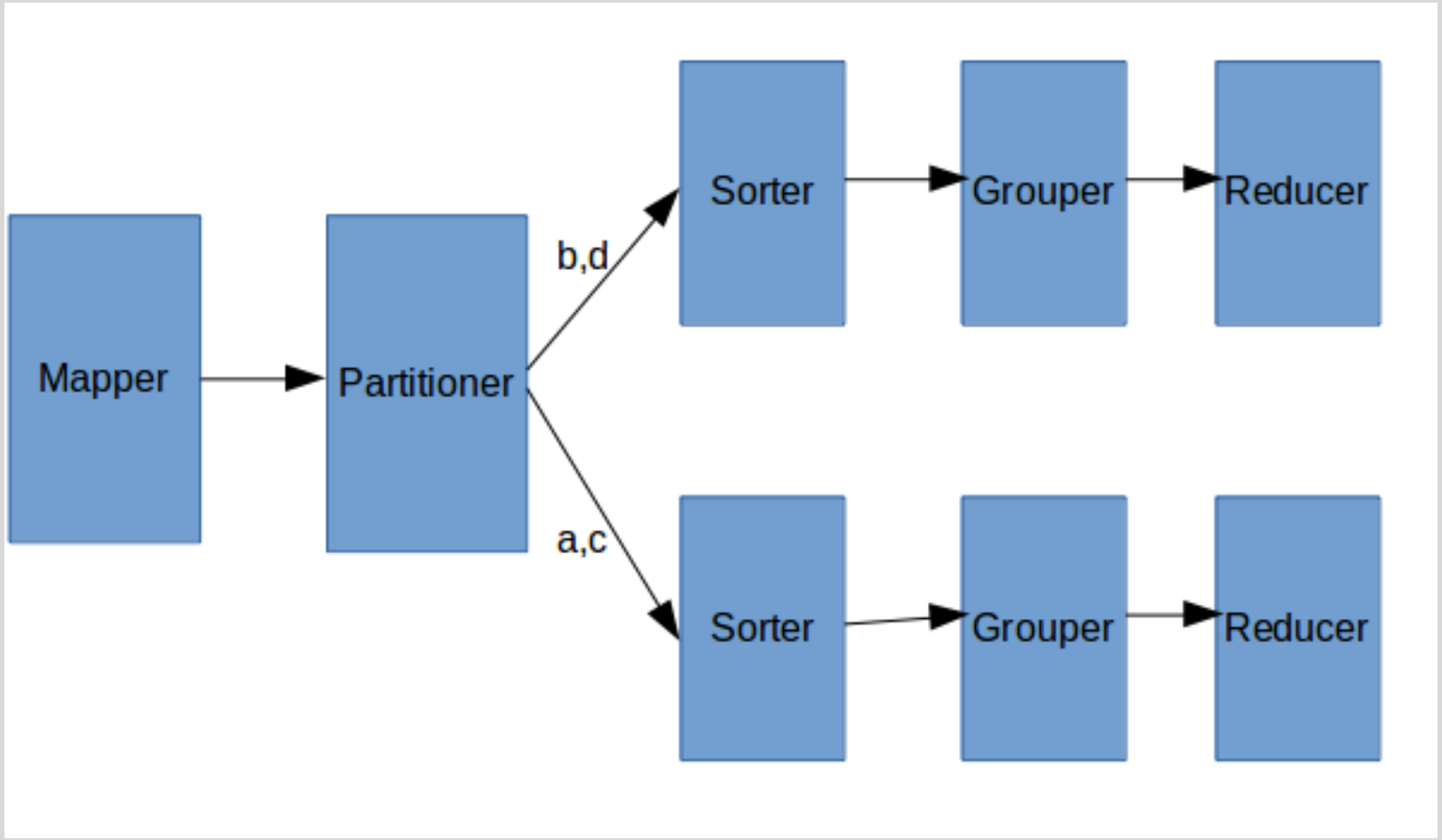
Secondary Sort – Implementation [Grouping Comparator]

Grouping Comparator

- Uses “natural” key only.
- Determines which keys and values are passed in a single call to the Reducer.

```
let compare(Pair k1, Pair k2) =
    compare k1.getPrimaryKey(), k2.getPrimaryKey()
```

Workflow of Custom Keys, Sorting and Grouping Data



https://www.kamalsblog.com/2017/08/hadoop-custom-keys-sorting-and-grouping.html

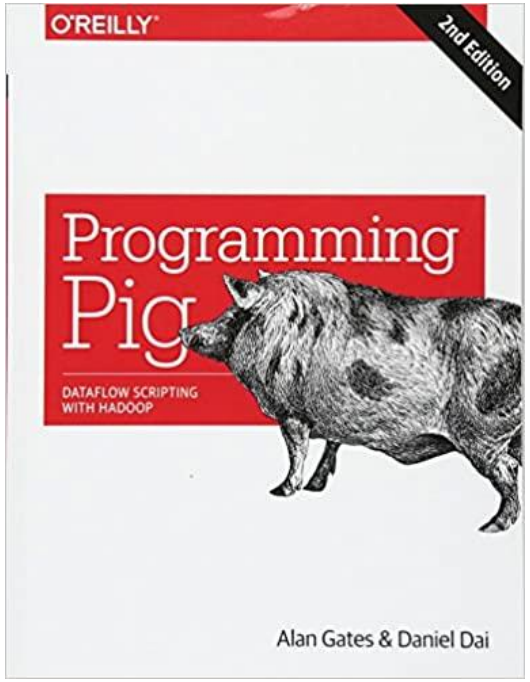
Secondary Sort – Implementation [Setting Comparators]

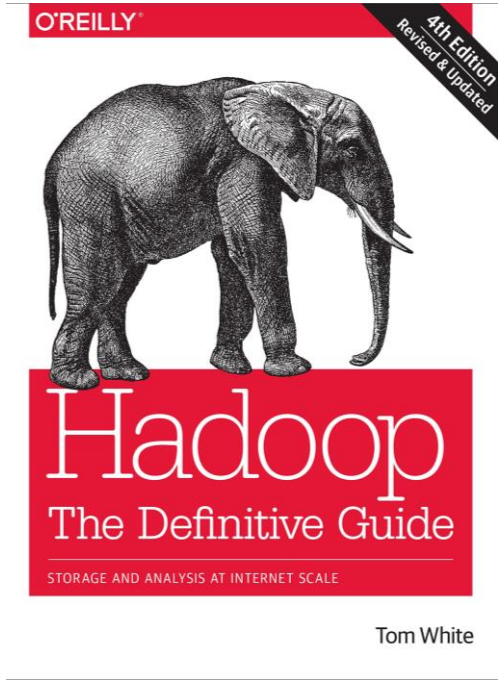
- Configure the job to use both comparators or either one of them.

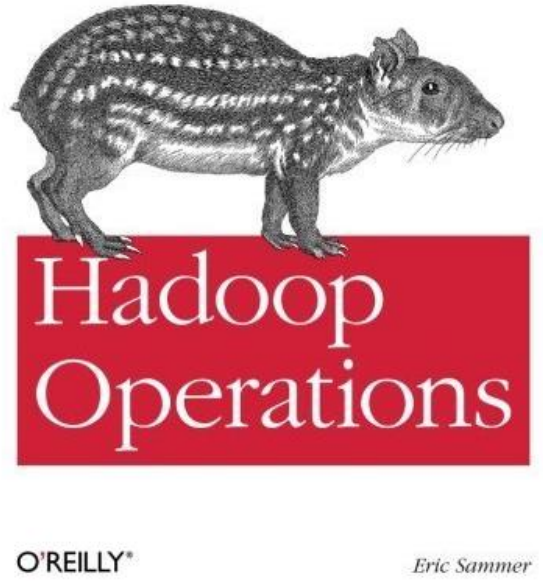
```
public class MyDriver extends Configured implements Tool {

    public int run(String[] args) throws Exception {
        …
        job.setSortComparatorClass(NameYearComparator.class);
        job.setGroupingComparatorClass(NameComparator.class);
        …
    }
}
```

- You can practice with the “secondarysort” project in the VM. Remember to unmark corresponding commands in the driver code before compiling it.







Texts and Resources

- Unless stated otherwise, the materials presented in this lecture are taken from:
  - Hadoop: the definitive guide, White, Tom (Tom E.) author., 4th ed.., Sebastopol, California: O'Reilly, 2015.
  - Hadoop operations, Sammer, Eric., Loukides, Michael Kosta.; Nash, Courtney.; Romano, Robert (Illustrator), illustrator., First edition., Sebastopol, CA: O'Reilly, 2012.
  - Programming pig: dataflow scripting with hadoop, Gates, Alan, author.; Dai, Daniel, 2nd edition., Beijing, China: O'Reilly, 2017

Unit Summary

- MapReduce Code Explanation.